7. Pipes, Redirection, and Exit Status in Pipelines

07. Pipes, Redirection, and Exit Status in Pipelines

Phase: Phase 3 — Text, pipes, and discovery
Estimated time: 1–2 days
Slug: `07-pipes-redirects`

Learning objectives

- Redirect stdout/stderr to files
- Connect commands with pipes
- Use tee, here-docs, and process substitution ideas
- Understand pipeline exit status (`pipefail`)

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics.

Redirection

```
cmd > out.txt        # stdout overwrite
cmd >> out.txt       # append
cmd 2> err.txt       # stderr
cmd > out.txt 2>&1   # both to same file
cmd < in.txt         # stdin from file
```

Pipes

```
ps aux | grep nginx | awk '{print $2}'
```

Data flows left → right as streams. Prefer tools that stream over saving giant temps.

Bash pipefail

By default, pipeline status is the last command. In scripts:

```
set -o pipefail
```

so failures in the middle are not hidden.

Common mistakes

- Clobbering important files with `>`
- Grepping `ps` for a process and matching the grep itself
- Ignoring stderr while debugging

Exercises

Work in `exercises/07-pipes-redirects/`. Run:

```
./test.sh
```

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** note one real command you ran on a Linux host related to this lesson
- **C (Challenge):** rewrite your solution from memory the next day

Do not open `solutions/07-pipes-redirects/` until you have an honest failing attempt.

Self-check

1. What does `2>&1` mean?
2. Why enable `pipefail`?
3. What does `tee` do?

Further reading

- https://www.gnu.org/software/bash/manual/html_node/Redirections.html